

Topic 3.9

3.9: Overloading the ++ Operator

Considering our [Distance](#) class, it is useful to overload the ++ operator. This is accomplished as follows:

```
class Distance
{
    ...
    // Define prefix ++ operator to add 1 foot
    // to a distance.

    Distance& operator++(void);
    ...
};

Distance& Distance::operator++(void)
{
    ++feet;
    return *this;    // We NEED this!
}
```

Recall that ++ can be either a prefix or a postfix operator. By convention, above we have defined the prefix version of the operator. To define a postfix version of the operator, we declare the function to have a single [int](#) formal parameter. We are not required to make the two functions behave the same. Here, we define the postfix operator ++ to add one inch instead of one foot.

```
class Distance
{
    ...
    // Define postfix ++ operator to add 1 inch
    // to a distance.

    Distance& operator++(int x);
    ...
};

Distance& Distance::operator++(int x)
{
    inches++;
    return *this;    // We NEED this!
}
```

Note that in both cases the value produced by the operator refers to the *new* value after the incrementation. This is somewhat contradictory to what we normally think of for the postfix form of the operator. We can avoid this by creating a temporary [Distance](#) variable and returning it, not a reference to it, if we want:

```
Distance Distance::operator++(int x)
{
    Distance original=*this;
    inches++;
    return original;
}
```

Overloading Comparison Operators

We cannot compare two [Distance](#) variables because we do not have a comparison operator that knows how. But we can build one:

```
class Distance
{
    ...
    int operator==( Distance& rhs ) const;
    int operator<( Distance& rhs ) const;
    int operator>( Distance& rhs ) const;
    int operator<=( Distance& rhs ) const;
    int operator>=( Distance& rhs ) const;
    int operator!=( Distance& rhs ) const;
    ...
};

int Distance::operator==( Distance& rhs ) const
{
    return (feet == rhs.feet &&
            inches == rhs.inches);
}
```

We can overload the other comparison operators similarly.

By convention, these functions should return 1 if the comparison is true, 0 if the comparison is false.

It is wise if the operators `==` and `!=` are defined so that one is true exactly when the other is false. In other words, two items should never both be equal and not equal at the same time. Neither should they be defined so that two items are neither equal nor not equal.

Guidelines for Overloading Operators

- Use operator overloading when it aids program readability. Generally, this will happen if the operators you are overloading have commonly accepted meanings when applied to items of the new type being defined.
- Don't give operators counterintuitive meanings, for instance, `+` should *not* mean subtract.
- Operators generally should return an object of the class on which the operator is being overloaded, or a reference to such an object.

The following program illustrates overloading of the `++` operator:

```
// MODULE:      distance4.cpp
// PROGRAMMER: Gary J. Blair
// DATE:        4/11/1996
// COMPILER:    MS Visual C++ 2008 Express Edition
// REQUIRED:     distance4.cpp, dist4.h, dist4.cpp
// PURPOSE:
//   Provide a class for English distances.
//   Illustrate overloading of ++ operator.

#include <iostream>
#include <stdlib.h>
#include "dist4.h"

int main(void)
{
    Distance a(3,6.0);
    Distance b;

    b = a;

    cout << "Initially a=" << a << " and b=" << b
        << endl;
    ++a;
    b++;
    cout << "After preincrementing, a=" << a
        << endl;
    cout << "After postincrementing, b=" << b
        << endl;

    if (a == b)
        cout << a << " is equal to " << b << endl;
    else
        cout << a << " is not equal to " << b
            << endl;

    cout << a << " + " << b << " = ";
    a += b;
    cout << a << endl;

    Distance x(4,5.0);

    cout << x << endl;
    x.operator++();
    cout << x << endl;
    x.operator++(3);
    cout << x << endl;
}
```

Output for distance4.cpp:

```
Initially a=3'6" and b=3'6"
After preincrementing, a=4'6"
After postincrementing, b=3'7"
4'6" is not equal to 3'7"
4'6" + 3'7" = 8'1"
4'5"
5'5"
5'6"
```

```
// MODULE      dist4.h
// PROGRAMMER: Gary J. Blair
```

```
// DATE:      4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    distance4.cpp, dist4.h, dist4.cpp
// PURPOSE:
//   Header file for Distance class.

using namespace std;

class Distance
{
    public:
        // The no argument constructor initializes
        // a Distance to 0'0".  Distance();

        // Initialize a Distance to the specified value.  Distance( int ft, double in );
        // Define + that operates on
        // Distance + Distance.  Distance operator + ( Distance &rhs ) const;

        // Define + that operates on Distance + int,
        // which means add feet.

        Distance operator + ( int rhs ) const;

        // Define + that operates on Distance + double,
        // which means add inches.

        Distance operator + ( double rhs ) const;

        // Define prefix ++ that adds 1 foot.

        Distance& operator ++(void);

        // Define postfix ++ that adds 1 inch.

        Distance& operator ++(int x);

        // Define comparison operator.

        int operator==( Distance& rhs ) const;

        // Define += operator which operates on
        // 2 distances.

        Distance& operator+=( Distance& rhs );

        // Define + that operates on int + Distance,
        // which means add feet.

        friend Distance operator + ( int lhs,
                                    Distance &rhs );

        // Define + that operates on double + Distance,
        // which means add inches.

        friend Distance operator + ( double lhs,
                                    Distance &rhs );

        // Define stream insertion operator
        // for Distances.

        friend ostream& operator << ( ostream& str,
                                       Distance& d );

        // Define stream extraction operator
        // for Distances.

        friend istream& operator >> ( istream& str,
                                       Distance& d );
```

```

    private:
    int    feet;
    double inches;

};

// MODULE      dist4.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    distance4.cpp, dist4.h, dist4.cpp
// PURPOSE:
// Implement the Distance class.

#include <iostream>
#include <stdlib.h>
#include "dist4.h"

// The no argument constructor
// initializes a Distance to 0'0".

Distance::Distance()
{
    feet = 0;
    inches = 0.0;
}

// Initialize a Distance to the specified value.

Distance::Distance( int ft, double in )
{
    feet = ft;
    inches = in;
}

// Define + that operates on
// Distance + Distance.

Distance Distance::operator + ( Distance &rhs ) const
{
    int    sum_feet;
    double sum_inches;

    sum_feet = feet + rhs.feet;
    sum_inches = inches + rhs.inches;
    if (sum_inches >= 12.0)
    {
        sum_inches -= 12.0;
        ++sum_feet;
    }
    return Distance(sum_feet,sum_inches);
}

// Define + that operates on Distance + int,
// which means add feet.

Distance Distance::operator + ( int rhs ) const
{
    return *this + Distance(rhs,0.0);
}

// Define + that operates on Distance + double,
// which means add inches.

Distance Distance::operator + ( double rhs ) const
{
    return *this + Distance(0,rhs);
}

```

```

    }

// Define prefix ++ that adds 1 foot.

Distance& Distance::operator ++(void)
{
    ++feet;
    return *this;    // We NEED this pointer here!
}

// Define postfix ++ that adds 1 inch.

Distance& Distance::operator ++(int x)
{
    inches++;
    return *this;    // We NEED this pointer here!
}

// Define comparison operator.

int Distance::operator==( Distance& rhs ) const
{
    return (feet == rhs.feet &&
            inches == rhs.inches);
}

// Define += operator which operates
// on 2 distances.

Distance& Distance::operator+=( Distance& rhs )
{
    feet += rhs.feet;
    inches += rhs.inches;
    if (inches >= 12.0)
    {
        inches -= 12.0;
        ++feet;
    }
    return *this;
}

// Define + that operates on int + Distance,
// which means add feet.

Distance operator + ( int lhs, Distance &rhs )
{
    int    sum_feet;
    double sum_inches;

    sum_feet = lhs + rhs.feet;
    sum_inches = rhs.inches;
    return Distance(sum_feet,sum_inches);
}

// Define + that operates on double + Distance,
// which means add inches.

Distance operator + ( double lhs, Distance &rhs )
{
    return rhs + lhs;
}

// Define stream insertion operator for Distances.

ostream& operator << ( ostream& str, Distance& d )
{
    str << d.feet << " " << d.inches << "\"";
}

```

```
    return str;
}

// Define stream extraction operator
// for Distances.

istream& operator >> ( istream& str, Distance& d )
{
    str >> d.feet;
    if (str.get() != '\n')
    {
        cerr << "*** Error extracting Distance.\n";
        exit(1);
    }
    str >> d.inches;
    if (str.get() != '\n')
    {
        cerr << "*** Error extracting Distance.\n";
        exit(1);
    }
    return str;
}
```

[Back to Module 3 \(https://onlinelearning.berkeley.edu/courses/2291660/pages/module-3\)](https://onlinelearning.berkeley.edu/courses/2291660/pages/module-3)